

DRS 010

Guide d'Installation Complet

Angular (Windows) · Django · PostgreSQL · Redis · Celery · Netmiko

Linux VM (Backend) ← → Windows (Frontend)

Sur ta VM Linux

- ✓ Django déjà installé
- ✗ PostgreSQL → Phase 1
- ✗ Redis → Phase 2
- ✗ Celery → Phase 3
- ✗ Projet Django → Phase 4
- ✗ Netmiko/NAPALM → Phase 7

Sur Windows

- ✗ Node.js LTS → Phase 5
- ✗ Angular CLI → Phase 5
- ✗ Projet Angular → Phase 6
- ✗ Connexion VM → Phase 6
- ✗ Test end-to-end → Phase 8

1

Installer PostgreSQL

Sur ta VM Linux — base de données principale

STEP 1.1 Installer PostgreSQL

```
sudo apt update
sudo apt install postgresql postgresql-contrib -y
```

STEP 1.2 Vérifier que PostgreSQL tourne

```
sudo systemctl status postgresql
# Tu dois voir : Active: active (running)
```

STEP 1.3 Créer la base de données et l'utilisateur

```
sudo -u postgres psql
```

Une fois dans psql, tape ces commandes une par une :

```
CREATE DATABASE drs010_db;
CREATE USER drs010_user WITH PASSWORD 'drs010pass';
ALTER ROLE drs010_user SET client_encoding TO 'utf8';
ALTER ROLE drs010_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE drs010_user SET timezone TO 'UTC';
GRANT ALL PRIVILEGES ON DATABASE drs010_db TO drs010_user;
\q
```

STEP 1.4 Vérifier la connexion

```
psql -U drs010_user -d drs010_db -h localhost
# Si tu vois drs010_db=> c'est bon !
# Tape \q pour quitter
```

✓ Phase 1 terminée : PostgreSQL est installé, la base drs010_db est créée et accessible.

2

Installer Redis

Sur ta VM Linux — broker de messages pour Celery

STEP 2.1 Installer Redis

```
sudo apt install redis-server -y
```

STEP 2.2 Démarrer et activer Redis au boot

```
sudo systemctl start redis-server
sudo systemctl enable redis-server
```

STEP 2.3 Tester Redis

```
redis-cli ping
# Réponse attendue : PONG
```

✓ Phase 2 terminée : Redis tourne sur localhost:6379 et répond PONG.

3

Créer le projet Django

Sur ta VM Linux — structure du projet DRS 010

STEP 3.1 Créer le dossier projet et l'environnement virtuel

```
mkdir ~/drs010 && cd ~/drs010
python3 -m venv venv
source venv/bin/activate
# Tu dois voir (venv) au début de la ligne
```

STEP 3.2 Installer tous les packages Python

```
pip install django djangorestframework
pip install psycopg2-binary
pip install celery redis
pip install django-cors-headers
pip install djangorestframework-simplejwt
pip install netmiko napalm
```

💡 psycopg2-binary est le connecteur PostgreSQL pour Python. Sans lui, Django ne peut pas parler à PostgreSQL.

STEP 3.3 Créer le projet Django

```
django-admin startproject automation_project .
# Le point . crée le projet dans le dossier courant
```

STEP 3.4 Créer les apps (micro-services)

```
python manage.py startapp backhaul
python manage.py startapp b2b
python manage.py startapp services
python manage.py startapp provisioning
python manage.py startapp monitoring
python manage.py startapp reporting
```

Ton dossier doit ressembler à ça :

```
drs010/
├── venv/
├── automation_project/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── backhaul/
├── b2b/
├── services/
├── provisioning/
├── monitoring/
├── reporting/
└── manage.py
```

4

Configurer Django

settings.py · PostgreSQL · Celery · CORS

STEP 4.1 Ouvrir settings.py

```
nano automation_project/settings.py
# ou : code automation_project/settings.py (si VS Code installé)
```

STEP 4.2 Modifier INSTALLED_APPS

Trouve le bloc INSTALLED_APPS et remplace-le par :

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Third-party
    'rest_framework',
    'corsheaders',
    # DRS 010 apps
    'backhaul',
    'b2b',
    'services',
    'provisioning',
    'monitoring',
    'reporting',
]
```

STEP 4.3 Configurer PostgreSQL

Trouve le bloc DATABASES et remplace-le par :

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'drs010_db',
        'USER': 'drs010_user',
        'PASSWORD': 'drs010pass',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

STEP 4.4 Ajouter CORS, Celery et JWT

Ajoute ces lignes à la fin du fichier settings.py :

```
# — CORS (autorise Angular Windows à parler à Django VM) —
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # doit être EN PREMIER
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

```
]

CORS_ALLOWED_ORIGINS = [
    'http://localhost:4200',    # Angular dev server (Windows)
]
CORS_ALLOW_CREDENTIALS = True

# — Celery + Redis —
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'

# — Django REST Framework —
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

STEP 4.5 Tester la connexion PostgreSQL

```
python manage.py migrate
# Tu dois voir une liste de OK sans erreur
```

 Si tu vois 'role drs010_user does not exist' → tu as peut-être fait une faute dans la Phase 1. Refais Step 1.3.

STEP 4.6 Créer un superuser Django

```
python manage.py createsuperuser
# Entre un username, email, et mot de passe
```

5

Configurer Celery

Sur ta VM Linux — tâches asynchrones et planificateur

STEP 5.1 Créer le fichier celery.py

```
nano automation_project/celery.py
```

Colle ce contenu :

```
import os
from celery import Celery
from celery.schedules import crontab

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
                      'automation_project.settings')

app = Celery('automation_project')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

# Tâches planifiées automatiques
app.conf.beat_schedule = {
    'rapport-mensuel': {
        'task': 'reporting.tasks.generate_monthly_report',
        'schedule': crontab(day_of_month=1, hour=8, minute=0),
    },
    'supervision-liens': {
        'task': 'monitoring.tasks.check_all_links',
        'schedule': crontab(minute='*/5'), # toutes les 5 min
    },
}
```

STEP 5.2 Modifier automation_project/__init__.py

```
nano automation_project/__init__.py
```

Ajoute ces 2 lignes :

```
from .celery import app as celery_app
__all__ = ('celery_app',)
```

STEP 5.3 Créer une tâche de test dans backhaul

```
nano backhaul/tasks.py

from celery import shared_task

@shared_task
def test_task():
    return 'Celery fonctionne correctement !'
```

STEP 5.4 Tester Celery

Ouvre 2 terminaux sur ta VM :

Terminal 1 — Worker Celery :

```
cd ~/drs010 && source venv/bin/activate
celery -A automation_project worker --loglevel=info
```

Terminal 2 — Tester la tâche :

```
cd ~/drs010 && source venv/bin/activate
```

```
python manage.py shell
>>> from backhaul.tasks import test_task
>>> test_task.delay()
# Dans Terminal 1 tu dois voir la tâche s'exécuter
```

✓ Phase 5 terminée : Celery reçoit et exécute des tâches depuis Django via Redis.

6

Installer Angular sur Windows

Sur Windows — interface utilisateur

STEP 6.1 Installer Node.js

Va sur <https://nodejs.org> et télécharge la version LTS. Lance l'installateur .msi et laisse tout par défaut.

Vérifier l'installation (PowerShell) :

```
node --version    # ex: v20.x.x
npm --version     # ex: 10.x.x
```

STEP 6.2 Installer Angular CLI

```
npm install -g @angular/cli
ng version    # vérifie l'installation
```

STEP 6.3 Créer le projet Angular

```
cd C:\Users\TonNom\Documents
ng new drs010-frontend
# ? Would you like to add Angular routing? → Yes
# ? Which stylesheet format? → CSS

cd drs010-frontend
```

STEP 6.4 Installer Angular Material

```
ng add @angular/material
# Choisis le thème que tu préfères (ex: Indigo/Pink)
# Set up global Angular Material typography? → Yes
# Set up browser animations? → Yes
```

STEP 6.5 Activer HttpClient dans app.config.ts

```
# Ouvre : src/app/app.config.ts

import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(), // ← ajoute cette ligne
```

```
]
};
```

STEP 6.6 Créer le service API

```
ng generate service services/api
```

Ouvre `src/app/services/api.service.ts` et remplace le contenu :

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

// ⚠ Remplace par l'IP de ta VM (voir Phase 7)
export const API_URL = 'http://VM_IP:8000/api';

@Injectable({ providedIn: 'root' })
export class ApiService {
  constructor(private http: HttpClient) {}

  // Test de connexion
  ping() {
    return this.http.get(`${API_URL}/ping/`);
  }

  getBackhaulLinks() {
    return this.http.get(`${API_URL}/backhaul/links/`);
  }
}
```

✅ Phase 6 terminée : Angular est installé sur Windows avec HttpClient et Angular Material.

7

Connecter Windows ↔ VM Linux

Faire parler Angular et Django

STEP 7.1 Trouver l'IP de ta VM

Sur ta VM Linux :

```
ip addr show
# Cherche une ligne comme :
# inet 192.168.x.x → c'est l'IP de ta VM

# OU plus simple :
hostname -I
```

💡 Si tu utilises VirtualBox en mode NAT : Settings → Network → Advanced → Port Forwarding. Ajoute : Host 127.0.0.1:8000 → Guest 10.0.2.15:8000. Dans ce cas utilise 127.0.0.1 comme IP.

STEP 7.2 Lancer Django sur toutes les interfaces

Sur ta VM, lance Django comme ça (pas localhost, mais 0.0.0.0) :

```
cd ~/drs010 && source venv/bin/activate
python manage.py runserver 0.0.0.0:8000
```

STEP 7.3 Mettre l'IP dans Angular

Dans src/app/services/api.service.ts, remplace VM_IP par l'IP trouvée en 7.1 :

```
export const API_URL = 'http://192.168.1.XX:8000/api';
//                ↑ ton IP de VM ici
```

STEP 7.4 Créer un endpoint de test dans Django

Dans automation_project/urls.py :

```
from django.contrib import admin
from django.urls import path
from django.http import JsonResponse

def ping(request):
    return JsonResponse({'status': 'ok', 'message': 'DRS 010 API running'})

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/ping/', ping),
]
```

STEP 7.5 Tester depuis Windows

Ouvre ton navigateur sur Windows et va sur :

```
http://192.168.1.XX:8000/api/ping/
# Tu dois voir : {"status": "ok", "message": "DRS 010 API running"}
```



Si la page ne charge pas : vérifie que le firewall de ta VM laisse passer le port 8000. Commande :

```
sudo ufw allow 8000
```

8

Tester Netmiko

Sur ta VM Linux — connexion aux équipements réseau

STEP 8.1 Vérifier l'installation

```
source venv/bin/activate
python -c "import netmiko; print('Netmiko OK')"
python -c "import napalm; print('NAPALM OK')"
```

STEP 8.2 Template de connexion Huawei

Voici le template que tu utiliseras quand tu auras les accès équipements (fournis par ta superviseure) :

```
from netmiko import ConnectHandler

device = {
    'device_type': 'huawei',
    'host': 'IP_EQUIPEMENT',
    'username': 'admin',
    'password': 'password',
    'port': 22,
}

with ConnectHandler(**device) as conn:
    output = conn.send_command('display version')
    print(output)
```



Ta superviseure te donnera les IPs et credentials des équipements lab. N'utilise jamais les vrais équipements de production sans validation.

9

Lancer l'environnement complet

La séquence à suivre à chaque session de travail

À chaque fois que tu commences à travailler, ouvre 3 terminaux sur ta VM et 1 PowerShell sur Windows :

VM — T1	Django API server <pre>cd ~/drs010 && source venv/bin/activate python manage.py runserver 0.0.0.0:8000</pre>
VM — T2	Celery Worker <pre>cd ~/drs010 && source venv/bin/activate celery -A automation_project worker --loglevel=info</pre>
VM — T3	Celery Beat (planificateur) <pre>cd ~/drs010 && source venv/bin/activate celery -A automation_project beat --loglevel=info</pre>
Windows	Angular Frontend (PowerShell) <pre>cd C:\Users\TonNom\Documents\drs010-frontend ng serve # Ouvre http://localhost:4200 dans le navigateur</pre>

✓ Checklist finale — tout est prêt quand :

✓	http://192.168.x.x:8000/api/ping/ répond {"status": "ok"}
✓	http://localhost:4200 affiche la page Angular dans le navigateur Windows
✓	Celery Worker affiche [ready] dans Terminal 2
✓	python manage.py migrate s'exécute sans erreur
✓	L'admin Django http://192.168.x.x:8000/admin/ est accessible
✓	redis-cli ping répond PONG